

TP n° 2

Requêtes SQL, Dépendances Fonctionnelles et Normalisation
Oracle APEX
Par DOUIRI Amjad

2026

Table des matières

1	Exercice n°1 : Requêtes SQL (Oracle APEX)	2
2	Exercice n°2 : Dépendances fonctionnelles et normalisation	6
3	Exercice n°3 : Système d'Armstrong et décompositions	7
4	Exercice n°4 : Algorithmes Python	8

1 Exercice n°1 : Requêtes SQL (Oracle APEX)

Adaptations Oracle : AS supprimé sur les tables dans FROM, EXCEPT remplacé par MINUS, toutes les chaînes entre guillemets simples.

Question 1 – Département avec le budget le plus élevé

```
1 SELECT dept_name
2 FROM department
3 WHERE budget = (SELECT max(budget) FROM department);
```

Question 2 – Enseignants gagnant plus que la moyenne

```
1 SELECT A.name, A.salary
2 FROM teacher A
3 WHERE A.salary > (SELECT avg(B.salary) FROM teacher B);
```

Question 3 – Étudiants ayant suivi ≥ 2 cours du même enseignant (avec HAVING)

```
1 SELECT teacher.name, student.name, COUNT(*) AS Total_Cours
2 FROM teacher
3 JOIN teaches ON teacher.ID = teaches.ID
4 JOIN takes ON (takes.course_id = teaches.course_id
5               AND takes.sec_id = teaches.sec_id
6               AND takes.semester = teaches.semester
7               AND takes.year = teaches.year)
8 JOIN student ON student.ID = takes.ID
9 GROUP BY teacher.name, student.name
10 HAVING COUNT(*) >= 2;
```

Question 4 – Même résultat sans HAVING

```
1 SELECT T.teachername, T.studentname, T.totalcount
2 FROM (
3     SELECT teacher.name AS teachername, student.name AS studentname
4           , COUNT(*) AS totalcount
5     FROM teacher
6     JOIN teaches ON teacher.ID = teaches.ID
7     JOIN takes ON (takes.course_id = teaches.course_id
8                   AND takes.sec_id = teaches.sec_id
9                   AND takes.semester = teaches.semester
10                  AND takes.year = teaches.year)
11     JOIN student ON student.ID = takes.ID
12     GROUP BY teacher.name, student.name
```

```
13 ) T
14 WHERE T.totalcount >= 2
15 ORDER BY T.teachername;
```

Question 5 – Étudiants sans cours avant 2010 (MINUS)

```
1 SELECT id, name FROM student
2 MINUS
3 SELECT student.id, student.name
4 FROM student
5 JOIN takes ON takes.id = student.id
6 WHERE takes.year < 2010;
```

Question 6 – Enseignants dont le nom commence par E

```
1 SELECT * FROM teacher WHERE name LIKE 'E%';
```

Question 7 – Quatrième salaire le plus élevé

```
1 SELECT name, salary
2 FROM teacher T1
3 WHERE 3 = (
4     SELECT COUNT(DISTINCT T2.salary)
5     FROM teacher T2
6     WHERE T2.salary > T1.salary
7 );
```

Question 8 – Les 3 salaires les moins élevés (ordre décroissant)

```
1 SELECT T1.name, T1.salary
2 FROM teacher T1
3 WHERE 2 >= (
4     SELECT COUNT(DISTINCT T2.salary)
5     FROM teacher T2
6     WHERE T2.salary < T1.salary
7 )
8 ORDER BY T1.salary DESC;
```

Question 9 – Étudiants en automne 2009 (IN)

```
1 SELECT S.name
2 FROM student S
3 WHERE ('Fall', 2009) IN (
4     SELECT semester, year FROM takes WHERE takes.id = S.id
5 );
```

Question 10 – Étudiants en automne 2009 (SOME)

```
1 SELECT S.name
2 FROM student S
3 WHERE ('Fall', 2009) = SOME (
4     SELECT semester, year FROM takes WHERE takes.id = S.id
5 );
```

Question 11 – Étudiants en automne 2009 (NATURAL JOIN)

```
1 SELECT DISTINCT student.name
2 FROM takes NATURAL JOIN student
3 WHERE takes.semester = 'Fall' AND takes.year = 2009;
```

Question 12 – Étudiants en automne 2009 (EXISTS)

```
1 SELECT name
2 FROM student
3 WHERE EXISTS (
4     SELECT 1 FROM takes
5     WHERE takes.id = student.id
6         AND semester = 'Fall'
7         AND year = 2009
8 );
```

Question 13 – Paires d'étudiants ayant suivi un cours ensemble

```
1 SELECT A.name, B.name
2 FROM (SELECT student.id, student.name,
3             takes.course_id, takes.sec_id,
4             takes.semester, takes.year
5         FROM student JOIN takes ON student.id = takes.id) A
6 JOIN (SELECT student.id, student.name,
7             takes.course_id, takes.sec_id,
8             takes.semester, takes.year
9         FROM student JOIN takes ON student.id = takes.id) B
10 ON A.course_id = B.course_id
11 AND A.sec_id = B.sec_id
12 AND A.semester = B.semester
13 AND A.year = B.year
14 AND A.id <> B.id
15 AND A.name < B.name
16 GROUP BY A.id, B.id, A.name, B.name
17 HAVING COUNT(*) >= 1;
```

Question 14 – Nombre d'étudiants par enseignant (ayant assuré un cours)

```
1 SELECT teacher.name, count(*) AS total_students
2 FROM takes
3 JOIN teaches ON takes.course_id = teaches.course_id
4               AND takes.sec_id      = teaches.sec_id
5               AND takes.semester    = teaches.semester
6               AND takes.year        = teaches.year
7 JOIN teacher ON teaches.id = teacher.id
8 GROUP BY teacher.name, teacher.id
9 ORDER BY total_students DESC;
```

Question 15 – Même chose, même sans cours assuré (LEFT JOIN)

```
1 SELECT teacher.name, count(takes.course_id) AS total_students
2 FROM teacher
3 LEFT JOIN teaches ON teacher.ID = teaches.ID
4 LEFT JOIN takes   ON teaches.course_id = takes.course_id
5                   AND teaches.sec_id   = takes.sec_id
6                   AND teaches.semester = takes.semester
7                   AND teaches.year     = takes.year
8 GROUP BY teacher.name, teacher.id
9 ORDER BY total_students DESC;
```

Question 16 – Nombre de grades A par enseignant (WITH)

```
1 WITH mytakes AS (
2     SELECT id, course_id, sec_id, semester, year, grade
3     FROM takes WHERE grade = 'A'
4 )
5 SELECT teacher.name, COUNT(mytakes.course_id) AS a_grades
6 FROM teacher
7 LEFT JOIN teaches ON teacher.ID = teaches.ID
8 LEFT JOIN mytakes ON teaches.course_id = mytakes.course_id
9                   AND teaches.sec_id   = mytakes.sec_id
10                  AND teaches.semester = mytakes.semester
11                  AND teaches.year     = mytakes.year
12 GROUP BY teacher.name, teacher.id
13 ORDER BY a_grades DESC;
```

Question 17 – Toutes les paires enseignant-étudiant + nombre de cours

```
1 SELECT teacher.name AS Teacher_Name,
2        student.name AS Student_Name,
3        COUNT(*) AS courses_taken
```

```

4 FROM teacher
5 JOIN teaches ON teacher.ID = teaches.ID
6 JOIN takes   ON teaches.course_id = takes.course_id
7             AND teaches.sec_id     = takes.sec_id
8             AND teaches.semester   = takes.semester
9             AND teaches.year       = takes.year
10 JOIN student ON takes.ID = student.ID
11 GROUP BY teacher.name, student.name;

```

Question 18 – Paires où l'étudiant a suivi ≥ 2 cours de l'enseignant

```

1 SELECT mytable.tn AS Teacher_Name, mytable.sn AS Student_Name
2 FROM (
3     SELECT teacher.name AS tn, student.name AS sn, COUNT(*) AS tc
4     FROM teacher
5     JOIN teaches ON teacher.ID = teaches.ID
6     JOIN takes   ON teaches.course_id = takes.course_id
7                 AND teaches.sec_id     = takes.sec_id
8                 AND teaches.semester   = takes.semester
9                 AND teaches.year       = takes.year
10    JOIN student ON takes.ID = student.ID
11    GROUP BY teacher.name, student.name
12 ) mytable
13 WHERE tc >= 2;

```

2 Exercice n°2 : Dépendances fonctionnelles et normalisation

1. $R(A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\}$

- Clé candidate : $\{A\}$ ($A^+ = \{A, B, C\}$)
- Forme normale : 2NF (transitivité $A \rightarrow B \rightarrow C$ viole la BCNF car B n'est pas super-clé)
- Décomposition BCNF :
 - $R_1(A, B)$ avec $F_1 = \{A \rightarrow B\}$
 - $R_2(B, C)$ avec $F_2 = \{B \rightarrow C\}$

2. $R(A, B, C)$, $F = \{A \rightarrow C, A \rightarrow B\}$

- Clé candidate : $\{A\}$ ($A^+ = \{A, B, C\}$)
- Forme normale : BCNF — le membre gauche de chaque dépendance (A) est une super-clé. Aucune décomposition nécessaire.

3. $R(A, B, C)$, $F = \{A, B \rightarrow C; C \rightarrow B\}$

- Clés candidates : $\{A, B\}$ et $\{A, C\}$

- **Forme normale** : 3NF (tous les attributs sont premiers), mais pas BCNF car $C \rightarrow B$ et C n'est pas super-clé.
- **Décomposition BCNF** :
 - $R_1(C, B)$ avec $F_1 = \{C \rightarrow B\}$
 - $R_2(A, C)$

3 Exercice n°3 : Système d'Armstrong et décompositions

1. Dériver 16+ dépendances de $F = \{A \rightarrow B, C; C, D \rightarrow E; B \rightarrow D; E \rightarrow A\}$

1. $A \rightarrow A$ (Réflexivité)
2. $A \rightarrow B$ (Décomposition)
3. $A \rightarrow C$ (Décomposition)
4. $B \rightarrow D$ (Directe)
5. $A \rightarrow D$ (Transitivité : $A \rightarrow B, B \rightarrow D$)
6. $A \rightarrow CD$ (Union : $A \rightarrow C, A \rightarrow D$)
7. $CD \rightarrow E$ (Directe)
8. $A \rightarrow E$ (Transitivité : $A \rightarrow CD, CD \rightarrow E$)
9. $E \rightarrow A$ (Directe)
10. $E \rightarrow B$ (Transitivité : $E \rightarrow A, A \rightarrow B$)
11. $E \rightarrow C$ (Transitivité : $E \rightarrow A, A \rightarrow C$)
12. $E \rightarrow D$ (Transitivité : $E \rightarrow B, B \rightarrow D$)
13. $E \rightarrow BC$ (Union : $E \rightarrow B, E \rightarrow C$)
14. $E \rightarrow CD$ (Union : $E \rightarrow C, E \rightarrow D$)
15. $A \rightarrow ABCDE$ (A est clé candidate)
16. $E \rightarrow ABCDE$ (E est aussi clé candidate)
17. $BD \rightarrow E$ (Augmentation : $B \rightarrow D$, donc $BD \rightarrow D; CD \rightarrow E$)

2. $R(A, B, C, D, E, F)$, $F = \{A \rightarrow B, C, D; B, C \rightarrow D, E; B \rightarrow D; D \rightarrow A\}$

(a) Fermetures

$$B^+ = \{B\} \xrightarrow{B \rightarrow D} \{B, D\} \xrightarrow{D \rightarrow A} \{A, B, D\} \xrightarrow{A \rightarrow BCD} \{A, B, C, D\} \xrightarrow{BC \rightarrow E} \{A, B, C, D, E\}$$

$$\{A, B\}^+ = \{A, B\} \xrightarrow{A \rightarrow BCD} \{A, B, C, D\} \xrightarrow{BC \rightarrow E} \{A, B, C, D, E\}$$

(b) $\{A, F\}$ est une super-clé

$$\{A, F\}^+ : A \rightarrow B, C, D \Rightarrow \{A, B, C, D, F\} \xrightarrow{BC \rightarrow E} \{A, B, C, D, E, F\} = R \quad \checkmark$$

(c) Décomposition BCNF

$B \rightarrow D$ viole la BCNF (B n'est pas super-clé de R).

Décomposition :

- $R_1(A, B, C, D, E)$ — clés candidates : $A, B, D \rightarrow$ en BCNF
- $R_2(B, F)$ — seule dépendance triviale $\rightarrow$ en BCNF

3. Décompositions de $R(A, B, C, D, E)$ **(a) $R_1(A, B, C)$ et $R_2(A, D, E)$: sans perte**

$R_1 \cap R_2 = \{A\}$. Or $A \rightarrow B, C$ est dans F^+ donc A détermine $R_1 - R_2$.

Décomposition sans perte d'information. ✓

(b) $R_1(A, B, C)$ et $R_2(C, D, E)$: avec perte

$R_1 \cap R_2 = \{C\}$. Or C ne détermine ni $\{A, B\}$ ni $\{D, E\}$ dans F .

Décomposition avec perte d'information. ✗

4 Exercice n°4 : Algorithmes Python

```

1 import itertools
2
3 def printDependencies(F):
4     for alpha, beta in F:
5         print("\t", alpha, "→", beta)
6
7 def printRelations(T):
8     for R in T:
9         print("\t", R)
10
11 def powerSet(inputset):
12     _result = []
13     for r in range(1, len(inputset) + 1):
14         _result += map(set, itertools.combinations(inputset, r))
15     return _result
16
17 def computeAttributeClosure(F, K):
18     K_plus, size = set(K), 0
19     while size != len(K_plus):
20         size = len(K_plus)
21         for alpha, beta in F:
22             if alpha.issubset(K_plus):
23                 K_plus.update(beta)
24     return K_plus
25
26 def computeDependenciesClosure(F):
27     R = set()
28     for alpha, beta in F:
29         R.update(alpha | beta)

```



```

30     F_plus = []
31     for K in powerSet(R):
32         for beta in powerSet(computeAttributeClosure(F, K)):
33             F_plus.append([K, beta])
34     return F_plus
35
36 def isDependency(F, alpha, beta):
37     return beta.issubset(computeAttributeClosure(F, alpha))
38
39 def isSuperKey(F, R, K):
40     return R.issubset(computeAttributeClosure(F, K))
41
42 def isCandidateKey(F, R, K):
43     if not isSuperKey(F, R, K):
44         return False
45     for A in K:
46         _K1 = set(K)
47         _K1.discard(A)
48         if isSuperKey(F, R, _K1):
49             return False
50     return True
51
52 def computeAllCandidateKeys(F, R):
53     return [K for K in powerSet(R) if isCandidateKey(F, R, K)]
54
55 def computeAllSuperKeys(F, R):
56     return [K for K in powerSet(R) if isSuperKey(F, R, K)]
57
58 def computeOneCandidateKey(F, R):
59     K = set(R)
60     while not isCandidateKey(F, R, K):
61         for A in K:
62             if isSuperKey(F, R, K.difference({A})):
63                 K.remove(A)
64                 break
65     return K
66
67 def isBCNFRelation(F, R):
68     for K in powerSet(R):
69         K_plus = computeAttributeClosure(F, K)
70         Y = K_plus.difference(K)
71         if not R.issubset(K_plus) and not Y.isdisjoint(R):
72             return False, [K, Y & R]
73     return True, [{}, {}]
74
75 def isBCNFRelations(F, T):
76     for R in T:
77         if not isBCNFRelation(F, R)[0]:
78             return False, R
79     return True, {}
80

```

```
81 def computeBCNFDecomposition(F, T):
82     OUT, size = list(T), 0
83     while size != len(OUT):
84         size = len(OUT)
85         for R in OUT:
86             _isR_BCNF, [alpha, beta] = isBCNFRelation(F, R)
87             if not _isR_BCNF:
88                 if alpha | beta not in OUT:
89                     OUT.append(alpha | beta)
90                 if R.difference(beta) not in OUT:
91                     OUT.append(R.difference(beta))
92             OUT.remove(R)
93             break
94     return OUT
```